

Submitted By	Domain	Project	Status
Lingam Manasa	Python Development	Project 2	✓ Completed

01 | Objective

Build an **Expense Tracker** where users enter expense amounts and categories. The program accumulates them and displays the **Total Spent**. This project teaches **Data Accumulation** — the foundation of financial software, banking systems, and any backend that processes numerical data in real-time.

02 | Key Concepts Applied

Concept	Description
<code>total = 0.0</code>	Accumulator initialized OUTSIDE the loop — preserves state (Anatomy of State)
<code>total += amount</code>	Accumulator pattern: State(new) = State(old) + Input
<code>try / except ValueError</code>	Defensive coding — The Digital Poka-Yoke (input validation)
<code>int('100')</code> vs <code>float(input())</code>	Type safety: '100'+50='10050' (disaster) vs 150.0 (truth)
<code>while True + break</code>	Continuous audit loop with kill switch / sentinel value
IPO Model	Input (amount) → Process (total += amount) → Output (FINAL TOTAL)

03 | Source Code

```
expense_tracker.py
"""
DecodeLabs - Python Project 2: Expense Tracker
Submitted by : Lingam Manasa | Batch 2026
Key Skills   : Accumulators, Math Ops, try/except, while loop
"""

expenses = [] # Store all individual expenses
total     = 0.0 # Accumulator - initialized OUTSIDE the loop

def add_expense(amount, category):
```

EXPENSE TRACKER

Python Project 2 — DecodeLabs Industrial Training Kit

BATCH 2026
decodelabs.tech

```
"""PROCESS: Accumulate expense into total and store in list."""
global total
total += amount          # total = total + new_expense
expenses.append({"amount": amount, "category": category})
print(f" Added: Rs.{amount:.2f} [{category}] | Running Total: Rs.{total:.2f}")

def view_expenses():
    """OUTPUT: Display all expenses and final total."""
    if not expenses:
        print(" No expenses recorded yet.")
        return
    print("\n " + "=" * 38)
    print("      EXPENSE REPORT - DecodeLabs")
    print(" " + "=" * 38)
    for i, exp in enumerate(expenses, start=1):
        print(f" {i:>2}. {exp['category']:<20} Rs.{exp['amount']:>8.2f}")
    print(" " + "-" * 38)
    print(f" {'TOTAL SPENT':<24} Rs.{total:>8.2f}")
    print(" " + "=" * 38)

def main():
    """Entry point - the gatekeeper."""
    print("=" * 42)
    print("      DecodeLabs Expense Tracker v1.0")
    print("      Submitted by: Lingam Manasa")
    print("=" * 42)

    while True:
        # Continuous Audit Loop
        print("\n Options:")
        print("      1. Add an expense")
        print("      2. View all expenses & total")
        print("      3. Exit")

        choice = input("\n Enter your choice (1/2/3): ").strip()

        if choice == "1":
            try:
                # THE GATEKEEPER - Defensive Coding
                amount = float(input(" Enter expense amount (Rs.): ").strip())
                if amount <= 0:
                    print(" Amount must be greater than 0.")
                    continue
            except ValueError:
                # Poka-Yoke: catches "ten", "abc" etc.
                print(" Invalid! Enter a number (e.g. 100, 50.5).")
                continue

            category = input(" Enter category (Food/Travel/etc): ").strip()
```

```
        if not category:
            category = "General"
        add_expense(amount, category)

    elif choice == "2":
        view_expenses()

    elif choice == "3":                # Kill Switch - Sentinel Value
        view_expenses()
        print("\n Goodbye! Save to file for persistence.")
        break
    else:
        print(" Invalid choice. Enter 1, 2, or 3.")

if __name__ == "__main__":
    main()
```

04 | How It Works

The **accumulator (total = 0.0)** is set outside the loop so it remembers every expense. Each entry goes through the **Gatekeeper (try/except)** — if the user types 'ten' instead of 10, the **ValueError** is caught and the program continues safely. The **kill switch (choice == '3')** triggers a final report and breaks the loop gracefully.

05 | Sample Output

Terminal output when program runs:

```
=====
DecodeLabs Expense Tracker v1.0
Submitted by: Lingam Manasa
=====

Options: 1. Add  2. View  3. Exit

Enter your choice: 1
Enter expense amount (Rs.): 100
Enter category: Food
Added: Rs.100.00 [Food] | Running Total: Rs.100.00

Enter your choice: 1
Enter expense amount (Rs.): ten
Invalid! Enter a number (e.g. 100, 50.5).

Enter your choice: 1
Enter expense amount (Rs.): 50
Enter category: Travel
```

EXPENSE TRACKER

Python Project 2 — DecodeLabs Industrial Training Kit

BATCH 2026
decodelabs.tech

Added: Rs.50.00 [Travel] | Running Total: Rs.150.00

Enter your choice: 3

```
=====
EXPENSE REPORT - DecodeLabs
=====
1. Food                Rs.  100.00
2. Travel              Rs.   50.00
-----
TOTAL SPENT            Rs.  150.00
=====
```

06 | Quality Standard Checklist

	Criteria	Implementation
✓	Stability	Handles 5+ transactions without crashing
✓	State	total initialized OUTSIDE the loop
✓	Defense	try/except catches ValueError on invalid input
✓	Control	Kill switch (choice 3) prints final total before exit

"Input. Process. Output. Truth. — Welcome to Backend Engineering."

— DecodeLabs